

Spring AI: Module 09

Model Context Protocol (MCP)

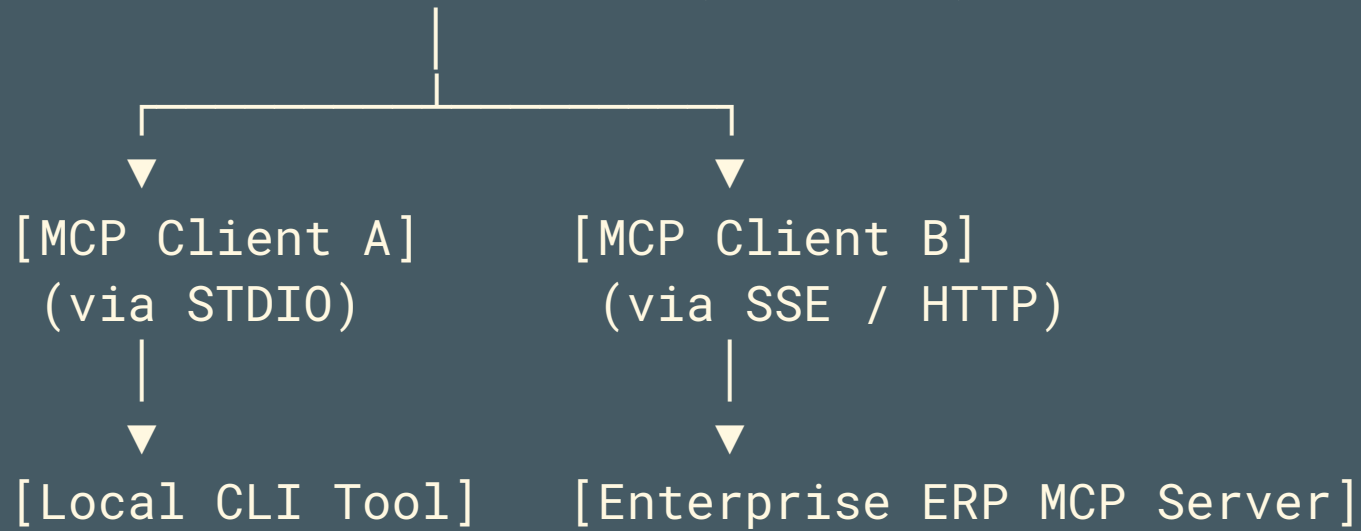
Building Enterprise MCP Servers & Clients

1. What is Model Context Protocol (MCP)?

- An open, standardized protocol by Anthropic enabling AI models to interact with external tools, prompts, and resources.
- Solves the $N \times M$ integration problem:
 - Instead of custom SDKs for every tool and model, tools implement MCP servers once.
 - Any MCP client can dynamically discover and call those tools.

2. Architecture: Host, Client, and Server

[Spring AI Application (MCP Host)]



3. Exposing Tools via Spring Boot MCP Server

```
public List<McpToolDescriptor> listTools() {  
    return List.of(new McpToolDescriptor(  
        "queryInventory",  
        "Queries real-time warehouse inventory for a given SKU",  
        Map.of("type", "object", "properties", Map.of("sku", Map.of("type", "string")))  
    ));  
}
```

4. MCP Security & Role-Based Access Control

- Never expose unrestricted MCP servers to untrusted environments.
- Implement strict permission layers:
 - Role-based tool access control (`ROLE_ADMIN`, `ROLE_STAFF`, `ROLE_GUEST`).
 - Argument validation and schema sanitization.
 - Audit logging for all tool executions.

5. Summary & Key Takeaways

1. MCP is becoming the industry standard for LLM-tool communication.
2. Spring Boot can act as both an MCP Server (exposing legacy enterprise systems) and an MCP Client (consuming external tools).
3. Sandbox tool execution and apply granular authorization to each MCP method.